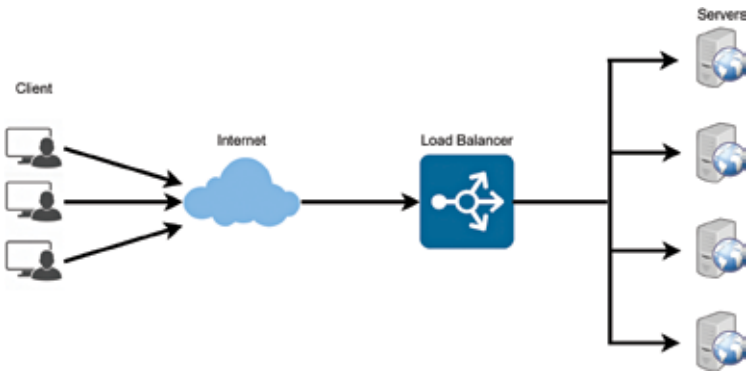


more machines to your pool of resources, whereas “vertical” scaling entails adding more power (e.g., CPU, RAM) to an existing system.

You nearly always want to scale horizontally in web development because, to put it simply, things break. Servers go down for no apparent reason. Networks deteriorate with time. Occasionally, entire data centres go down. Using many servers allows you to plan for outages and keep your application functioning. Your app is “defect tolerant,” in other terms.

Second, horizontal scalability allows you to decouple distinct components of your application’s backend (web server, database, service X, and so on) by running them on separate servers.



Finally, you may reach a point where vertical scaling is no longer practicable. There is no computer on the planet that can perform all of your app’s calculations. Consider Google’s search platform as a classic example, albeit this also applies to businesses on a much smaller scale. At any given time, massive websites are running 150-to-400 AWS EC2 instances, if they’re hosted on AWS. It would be difficult to deliver all of that compute capacity by vertical scaling.

Coming back to load balancers. They’re the secret sauce that allows you to scale horizontally. Incoming requests are routed to one of several application servers, which are often clones or mirror images of one another, and the app server’s answer is sent back to the client.

Any of them should handle the request, in the same way, therefore it’s only a matter of dividing the requests among the servers such that none of them gets overburdened. That is all there is to it. Load balancers are rather simple in concept.